

Reinforcement Learning

Policy Evaluation and Improvement using

Dynamic Programming

Debapriyo Majumdar
Indian Statistical Institute
debapriyo@isical.ac.in

Policy evaluation

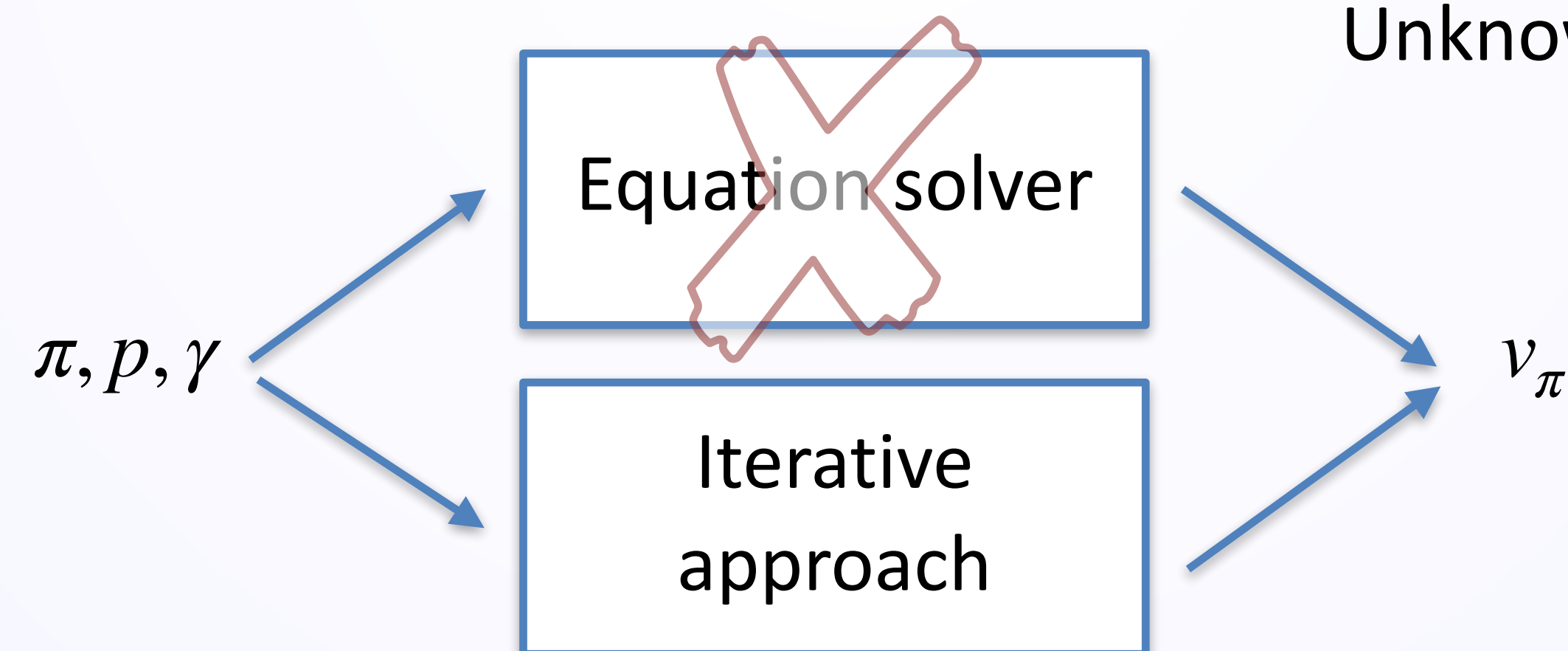
- We want to find a policy that would generate a high reward in the long run
- Policy evaluation: determine how good a policy π is
 - In other words, estimate or compute v_π
- Bellman equation:

If we know the dynamics of the environment (i.e., we know p)

$$v_\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \text{ for all } s \in \mathcal{S}$$

Unknown Unknown

We would have $|\mathcal{S}|$ linear equations in $|\mathcal{S}|$ unknowns



Iterative policy evaluation

- Initialize $v_0(s)$ arbitrarily for all $s \in \mathcal{S}$, and zero for the terminal state(s)
- Use the Bellman equation as an update rule:
$$v_{k+1}(s) := \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$
- Then, $v_k(s) \rightarrow v_\pi(s)$ as $k \rightarrow \infty$
 - *The proof uses the Banach Fixed Point Theorem / Contraction Mapping Theorem*
- Intuition: at each step, perform expected update
 - Expected immediate rewards from the old values of successor states
- Implementation: iteratively update and check the change Δ (maximum change in $v_k(s)$ over all states)
 - Stop when the change is $<$ some threshold θ

The Update Rule

$$v_{k+1}(s) := \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

$$= \sum_{a \in \mathcal{A}} \pi(a | s) \left[\sum_{s' \in \mathcal{S}, r \in \mathcal{R}} p(s', r | s, a) r + \sum_{s' \in \mathcal{S}, r \in \mathcal{R}} p(s', r | s, a) \gamma v_k(s') \right]$$

does not depend on s'

does not depend on r

$$= \sum_{a \in \mathcal{A}} \pi(a | s) \left[\sum_{r \in \mathcal{R}} p(r | s, a) r + \sum_{s' \in \mathcal{S}} p(s' | s, a) \gamma v_k(s') \right]$$

expected reward if we take action a from state s : denote by $r(s, a)$

$$= \sum_{a \in \mathcal{A}} \pi(a | s) \left[r(s, a) + \sum_{s' \in \mathcal{S}} p(s' | s, a) \gamma v_k(s') \right]$$

The Update Rule (continued)

- For state $s \in \mathcal{S}$,

$$v(s) := \sum_{a \in \mathcal{A}} \pi(a | s) \left[r(s, a) + \sum_{s' \in \mathcal{S}} p(s' | s, a) \gamma v(s') \right]$$

$$= r_s(\pi) + \gamma \sum_{s' \in \mathcal{S}} p_{s,s'}(\pi) v(s').$$

Define (the policy π is implicit here):

$$r_s = \sum_{a \in \mathcal{A}} \pi(a | s) r(s, a), \text{ and}$$

$$p_{s,s'} = \sum_{a \in \mathcal{A}} \pi(a | s) p(s' | s, a).$$

- The update rule in matrix form, together for all n states $1, 2, \dots, n$:

$$\begin{bmatrix} v(1) \\ v(2) \\ \vdots \\ v(n) \end{bmatrix} := \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_n \end{bmatrix} + \gamma \begin{bmatrix} p_{1,1} & \cdots & p_{1,n} \\ p_{2,1} & \cdots & p_{2,n} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,n} \end{bmatrix} \begin{bmatrix} v(1) \\ v(2) \\ \vdots \\ v(n) \end{bmatrix}$$

- Compactly, we can write $\mathbf{v} := \mathbf{r} + \gamma P \mathbf{v}$ or $\mathbf{v}^\pi := \mathbf{r}^\pi + \gamma P^\pi \mathbf{v}^\pi$ [under policy π]

The Policy Evaluation Algorithm

- Parameters: The dynamics P , convergence threshold $\theta > 0$

policy_evaluation(P, π, θ):

Initialize

$\mathbf{v} = (v_1, \dots, v_n)^T$ such that $v_T = 0$ and $v_i = \text{random}$ for all $i \neq T$

$\delta = +\infty$

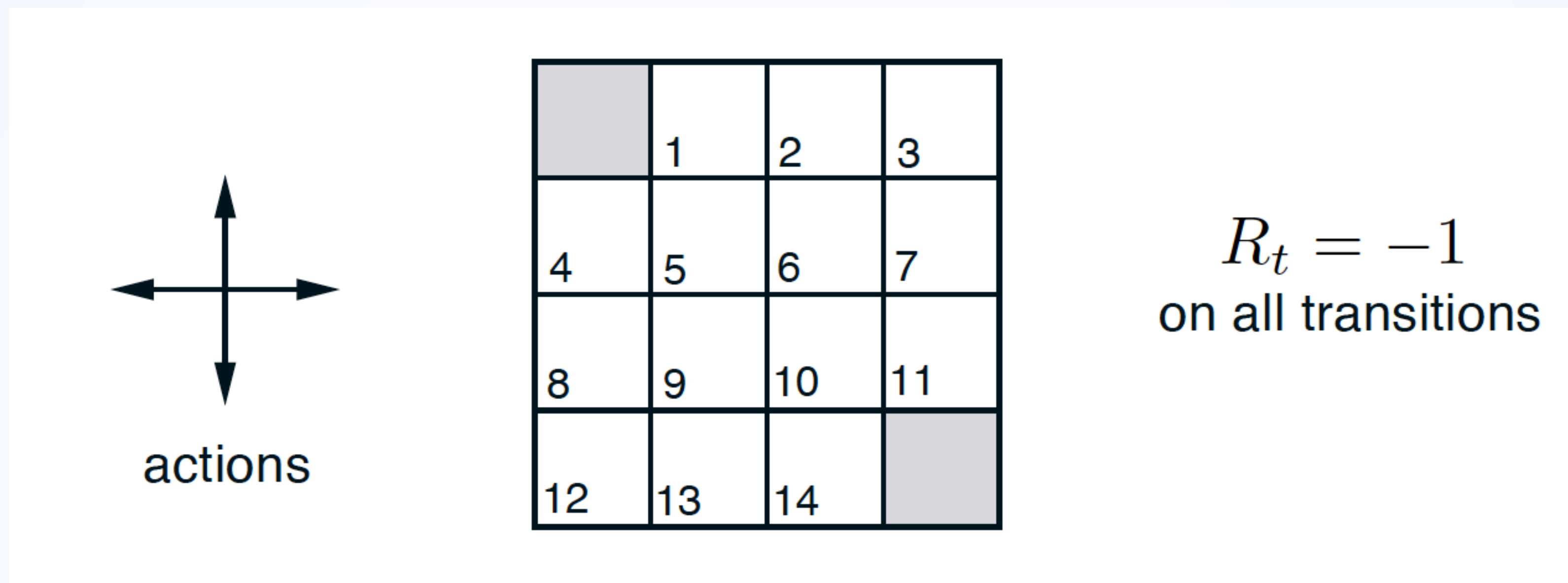
while $\delta > \theta$:

$\mathbf{v}_{\text{old}} := \mathbf{v}$

$\mathbf{v} := \mathbf{r} + \gamma P\mathbf{v}$ # updates values for all states

$\delta = \|\mathbf{v}_{\text{old}} - \mathbf{v}\|_{\infty}$

Example: Gridworld



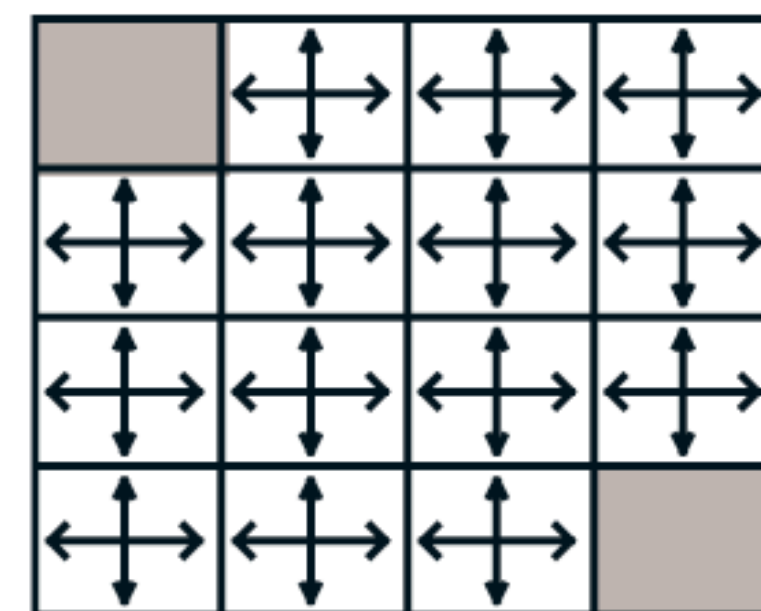
- Goal: get to the top-left or bottom-right corner (terminal states)
- Each step has reward -1 (should learn to get out of the grid faster)
- Undiscounted episodic task
- Environment:
 - $p(7, -1 | 2, \text{right}) = 0$ [impossible move]
 - $p(6, -1 | 5, \text{right}) = 1$ [only possible option], etc.

Example: Policy Evaluation on the Gridworld

v_k for the random policy

$k = 0$

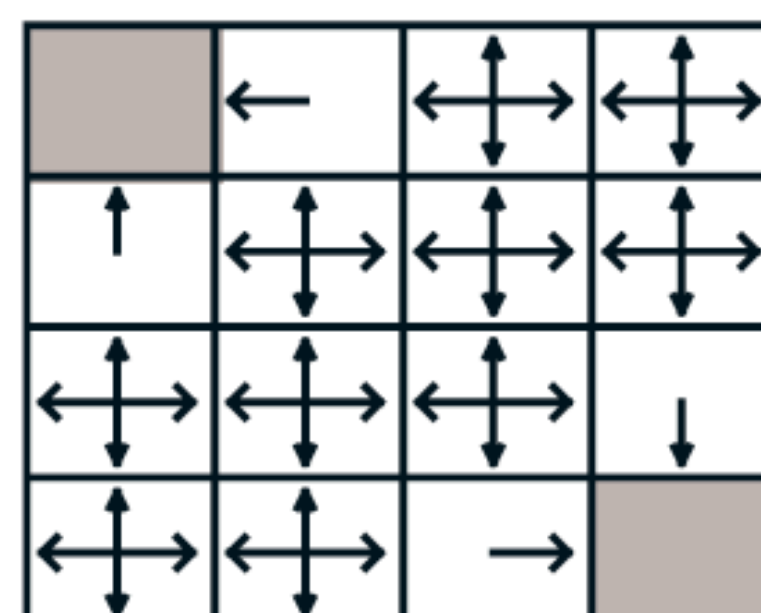
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0



← random policy

$k = 1$

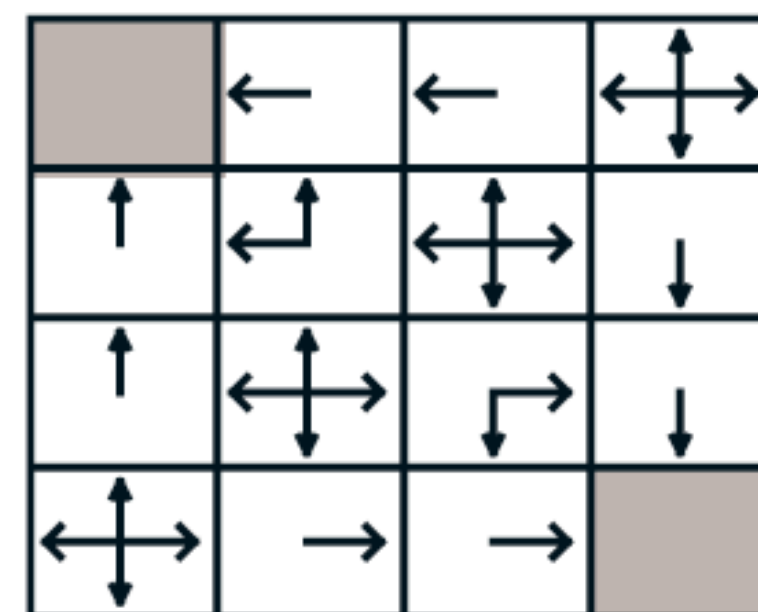
0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0



Example: Policy Evaluation on the Gridworld

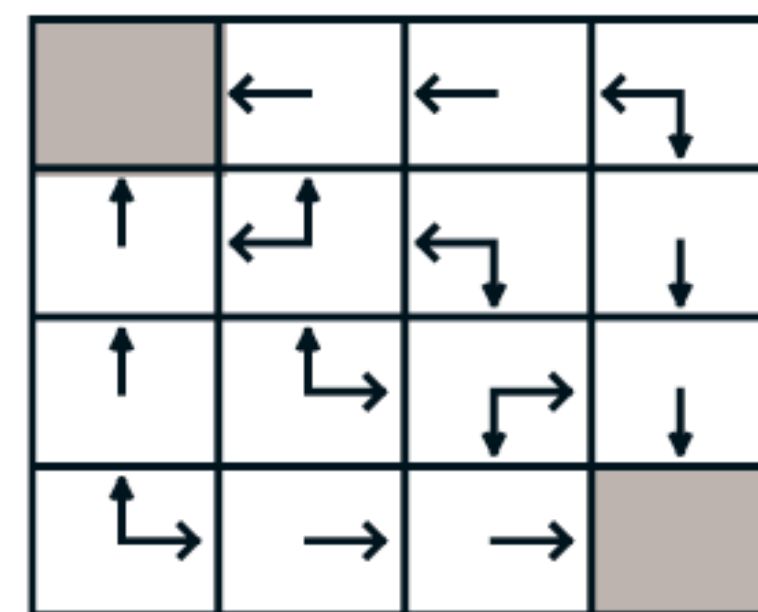
$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



$k = 3$

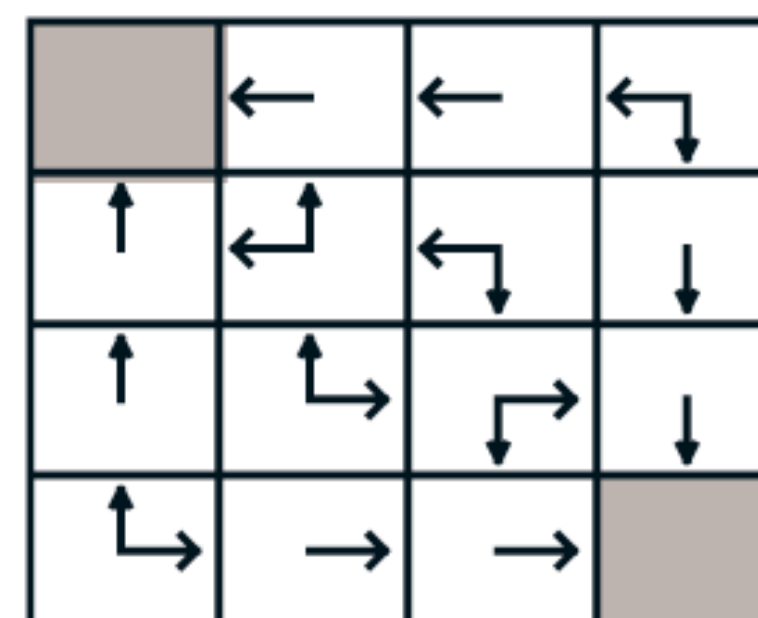
0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



Already
optimal policy
(the arrows are
not correct)

$k = \infty$

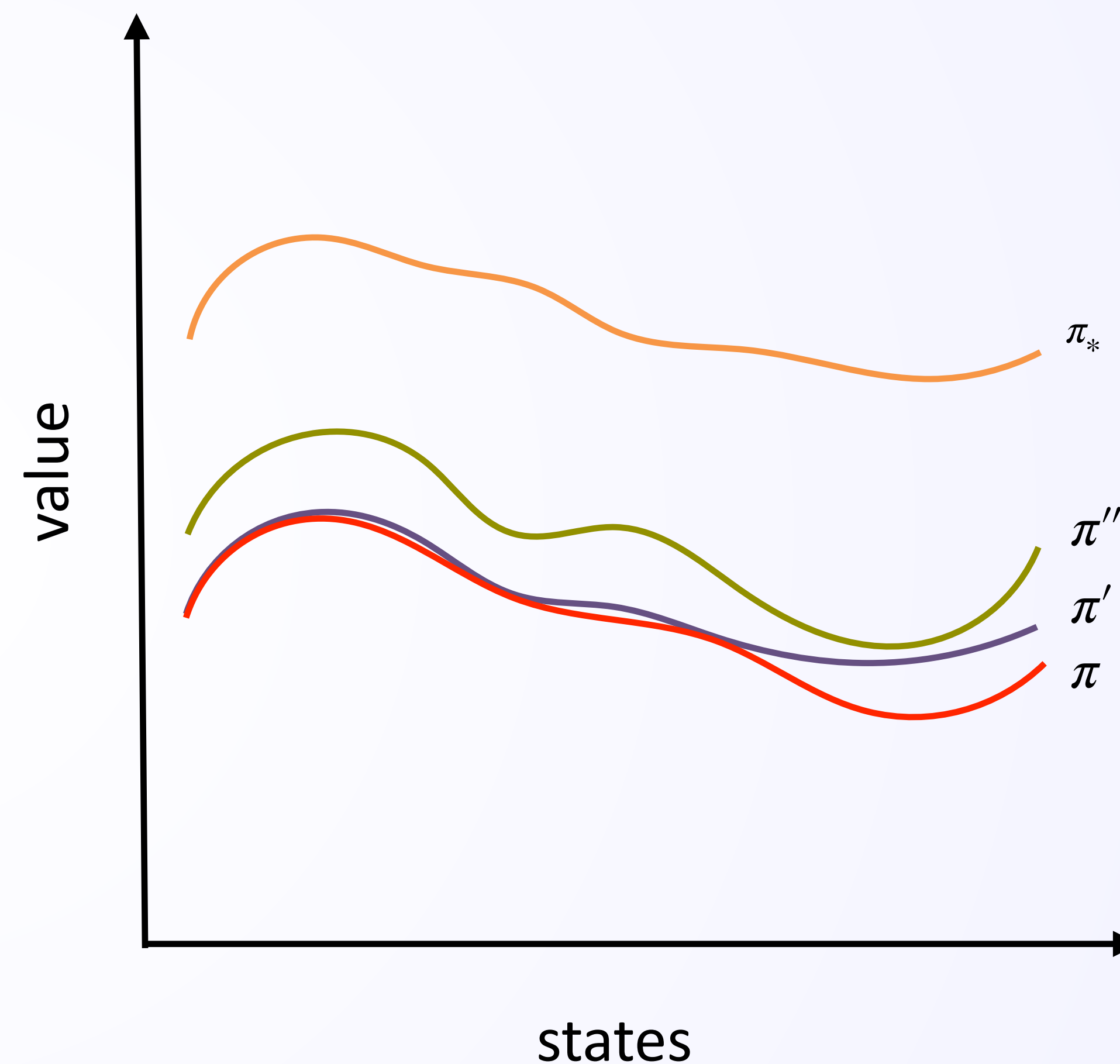
0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



Converged values

Policy improvement

- Recall the partial ordering between policies
- Order: $\pi \leq \pi'$ iff $v_\pi(s) \leq v_{\pi'}(s)$ for all $s \in \mathcal{S}$
- **Policy improvement** (or **policy control**): the process of improving a policy
- Approach: for every state iteratively improve the policy π
 - In other words, improve the value function monotonically
- When π does not improve for any state, it would have reached the optimal π_*
- Policy evaluation vs policy control
 - Policy control is essential for better learning
 - However, policy evaluation is a necessary step



Policy improvement theorem

Recall: $q_\pi(s, a)$ is the value (take action a and then follow policy π)

Value of taking the first action according to policy π' and then following policy π

Value of following policy π , in other words $= v_\pi(s)$

Suppose $q_\pi(s, \pi'(s)) \geq q_\pi(s, \pi(s))$ for all $s \in \mathcal{S}$

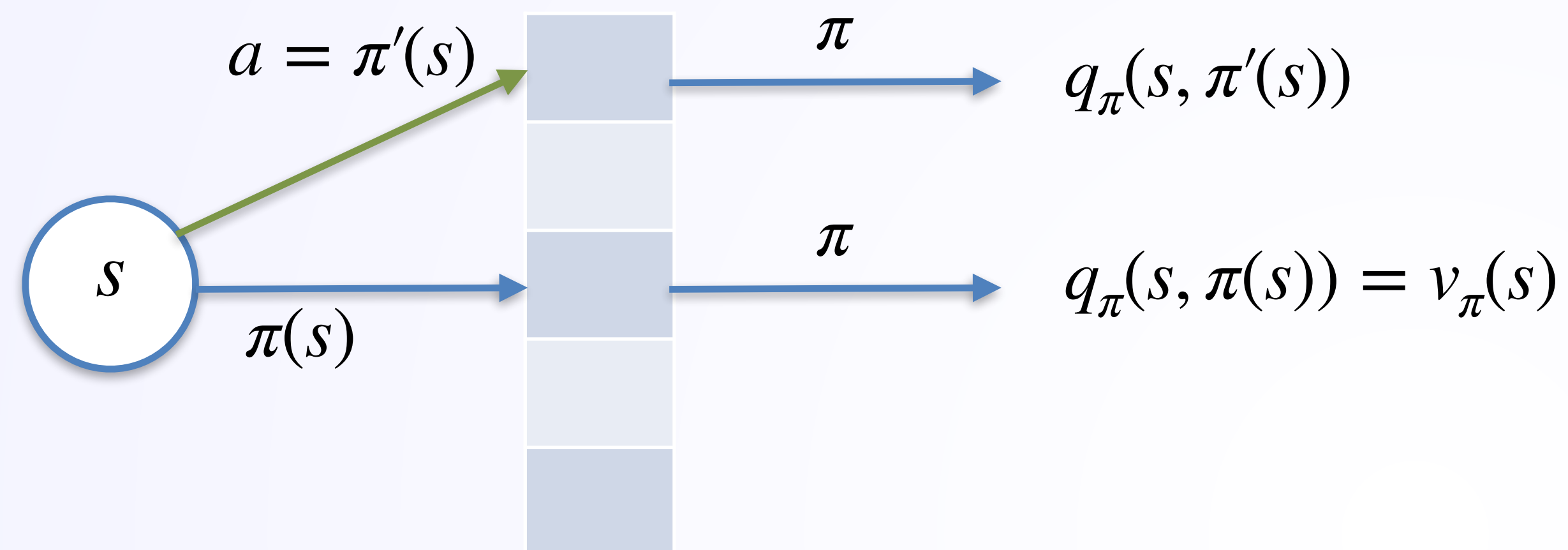
Then π' must be *better* than π

$$\pi' \geq \pi$$

Also, if $q_\pi(s, \pi'(s)) > q_\pi(s, \pi(s))$ for some $s \in \mathcal{S}$, then we can say $\pi' > \pi$

Greedy approach for policy improvement: from policy π , can we find another policy π' which is better?

Greedy improvement to a policy



If $q_\pi(s, a) > q_\pi(s, \pi(s))$ for some state s , then it would have been better to take action a
 $\implies$ can change the policy π

Greedy approach for policy improvement

$$\pi'(s) := \operatorname{argmax}_a q_\pi(s, a) \text{ for all states } s \in \mathcal{S}$$

Policy iteration

- The greedy approach defines a new policy, but the value function of that policy is still not known
- Approach: alternate between evaluation and improvement

- $$\pi_0 \xrightarrow{\text{evaluation}} v_{\pi_0} \xrightarrow{\text{improvement}} \pi_1 \xrightarrow{\text{evaluation}} v_{\pi_1} \xrightarrow{\text{improvement}} \dots \xrightarrow{\text{improvement}} \pi_* \xrightarrow{\text{evaluation}} v_{\pi_*}$$

- When the policy cannot be improved any further, then it has reached the optimal policy π_*

Policy Iteration

policy_iteration(P, θ):

Initialize: $\mathbf{v} = (v_1, \dots, v_n)^T$ such that $v_T = 0$, $v_i = \text{random } \forall i \neq T$, and $\pi(s) \in \mathcal{A}(s)$ arbitrarily $\forall s$

policy_stable := false

while (policy_stable == false)

 policy_evaluation(P, π, θ) # updates $\mathbf{v}$

 # Policy improvement

 policy_stable = true

 for each $s \in \mathcal{S}$:

 old_action := $\pi(s)$

$\pi(s) := \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{s'}]$

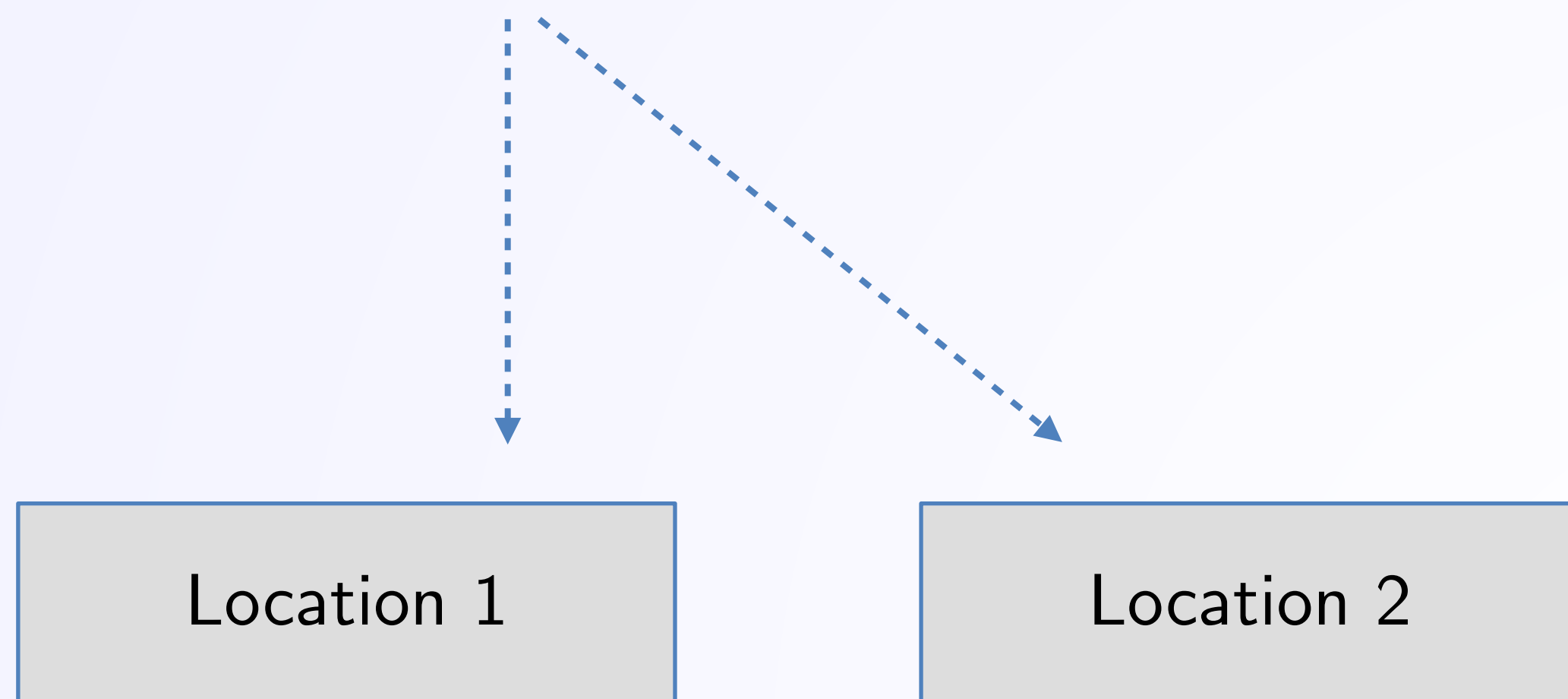
 if old_action $\neq \pi(s)$, then policy_stable = false

after exiting while loop

return $\mathbf{v}_* = \mathbf{v}$ and $\pi_* = \pi$

Car Rental Agency

Parent company



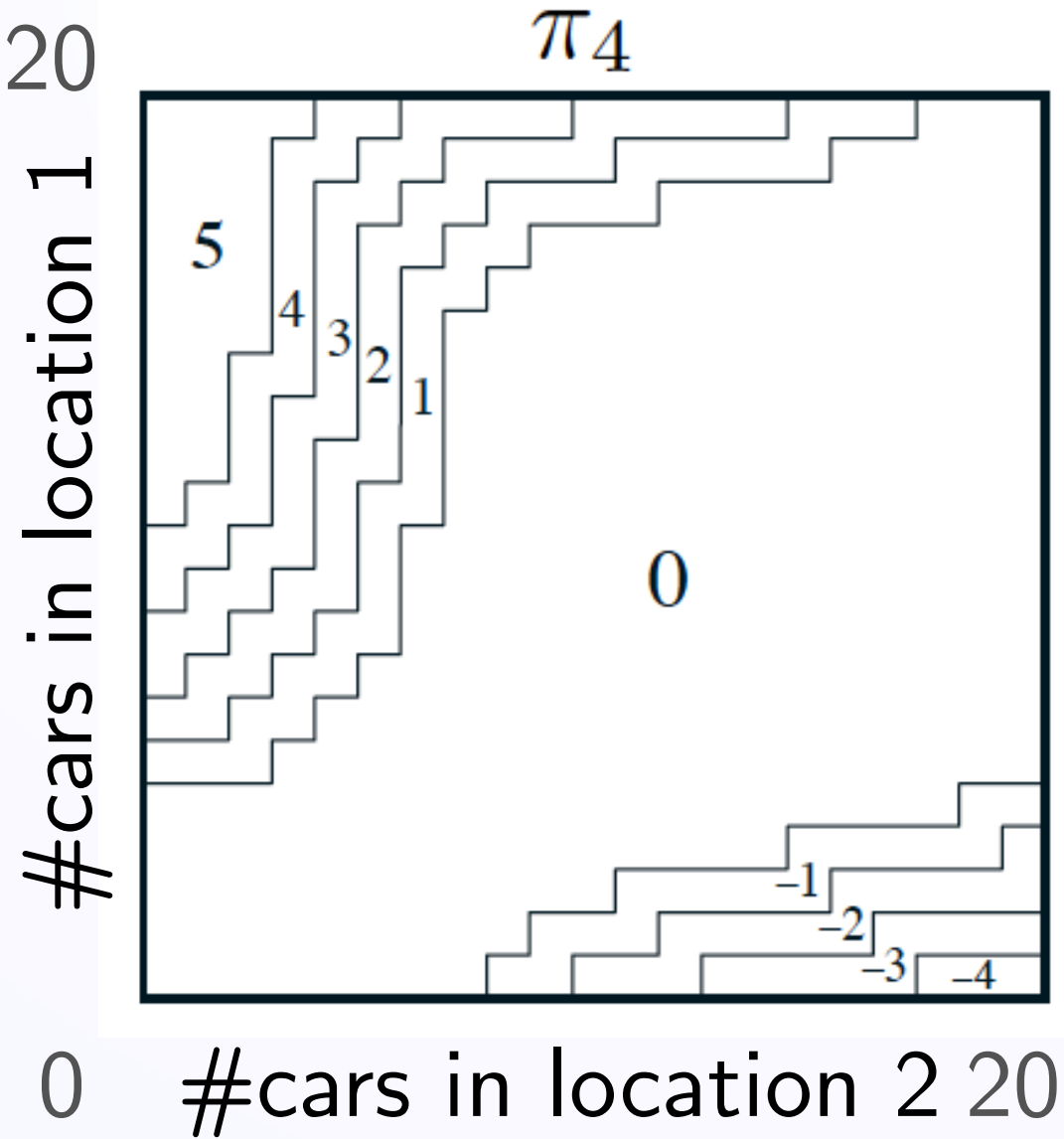
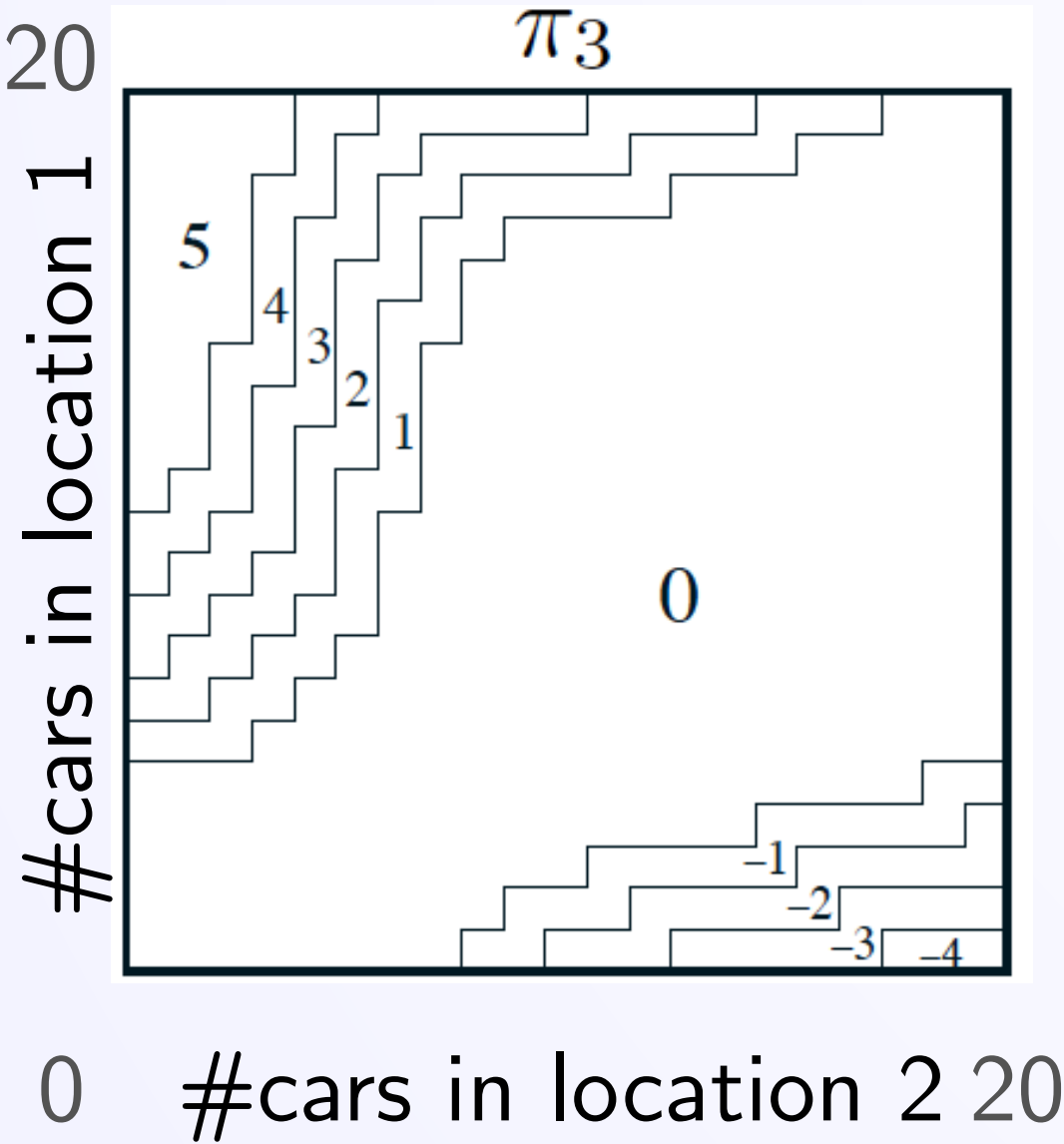
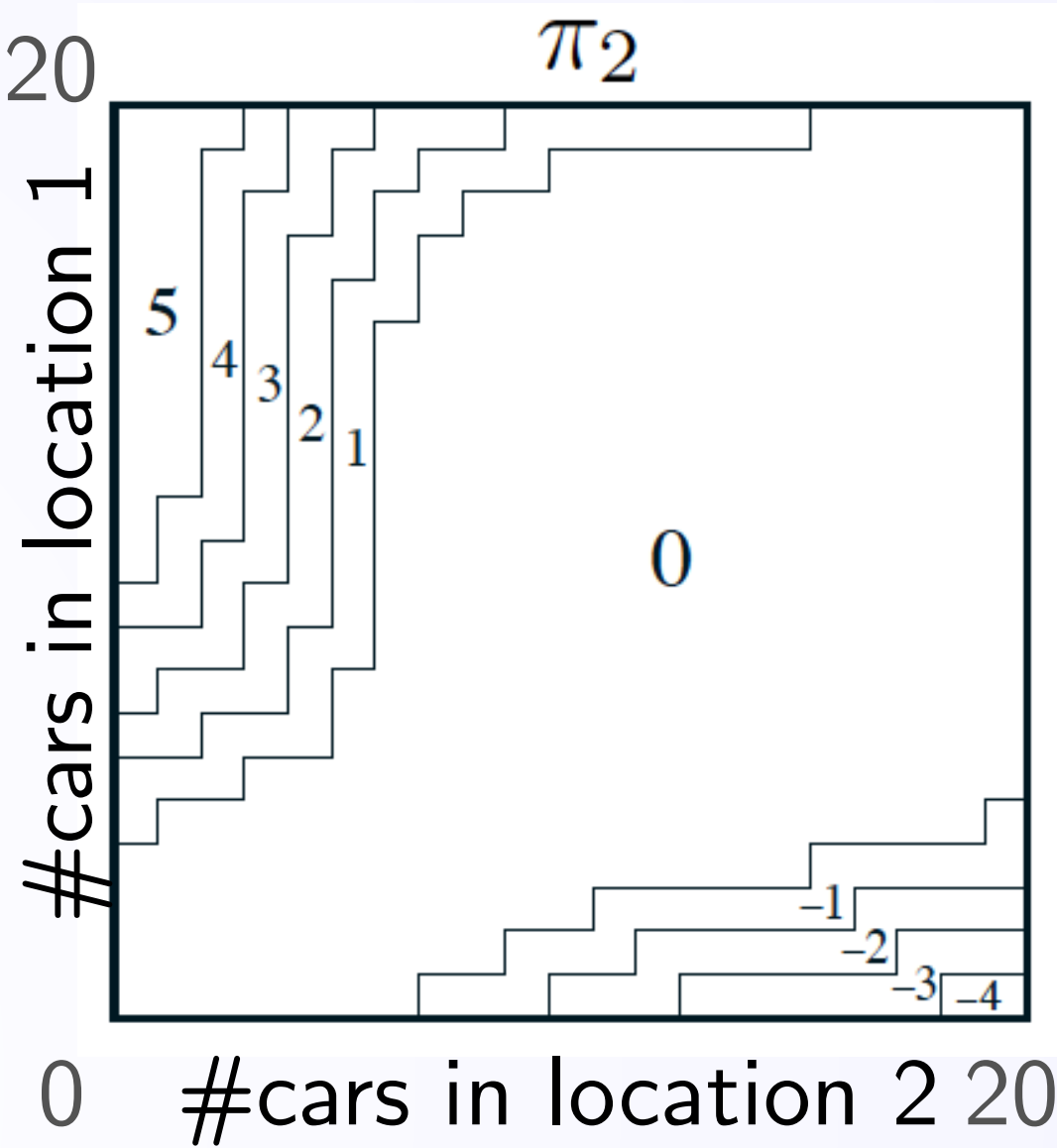
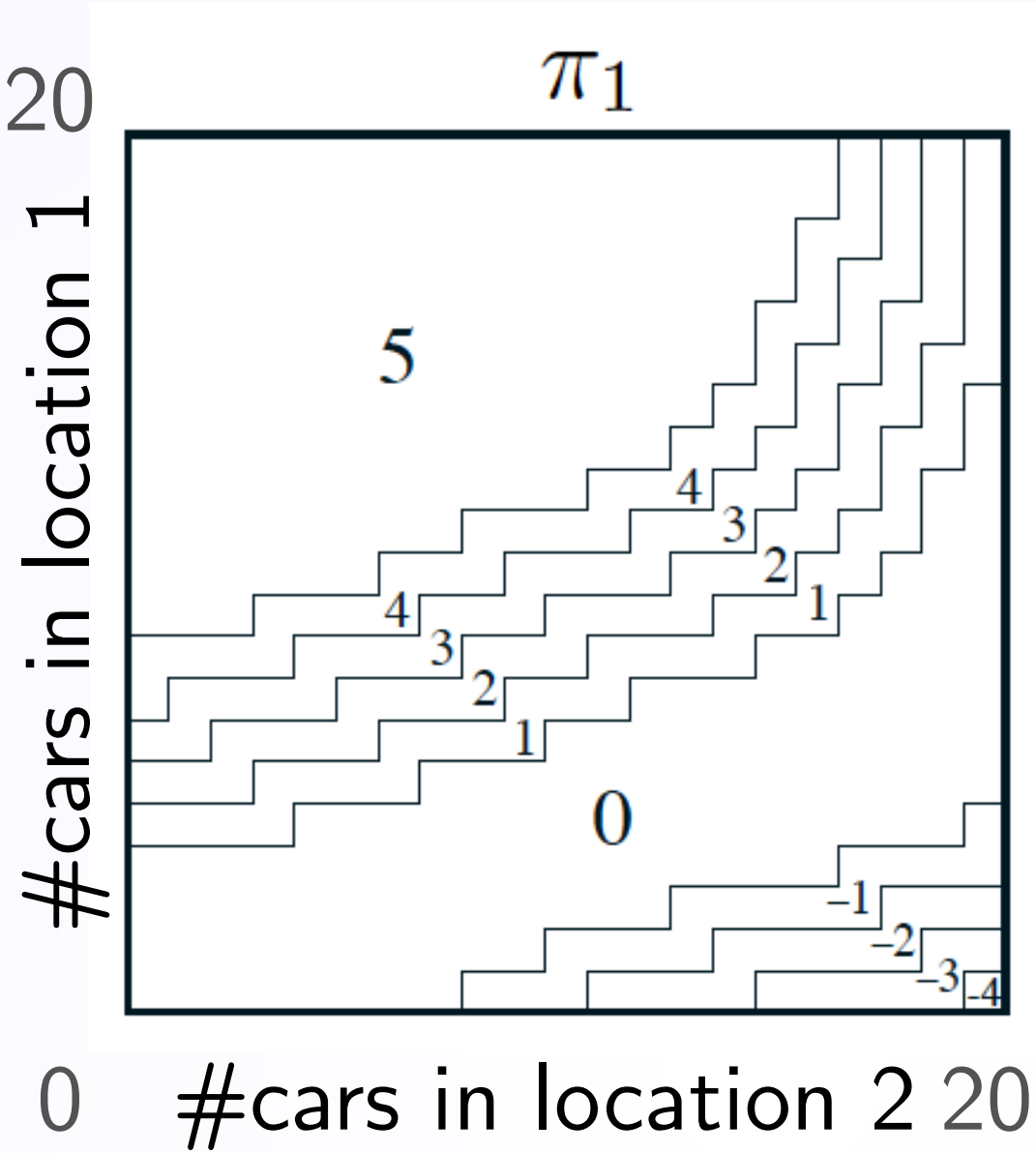
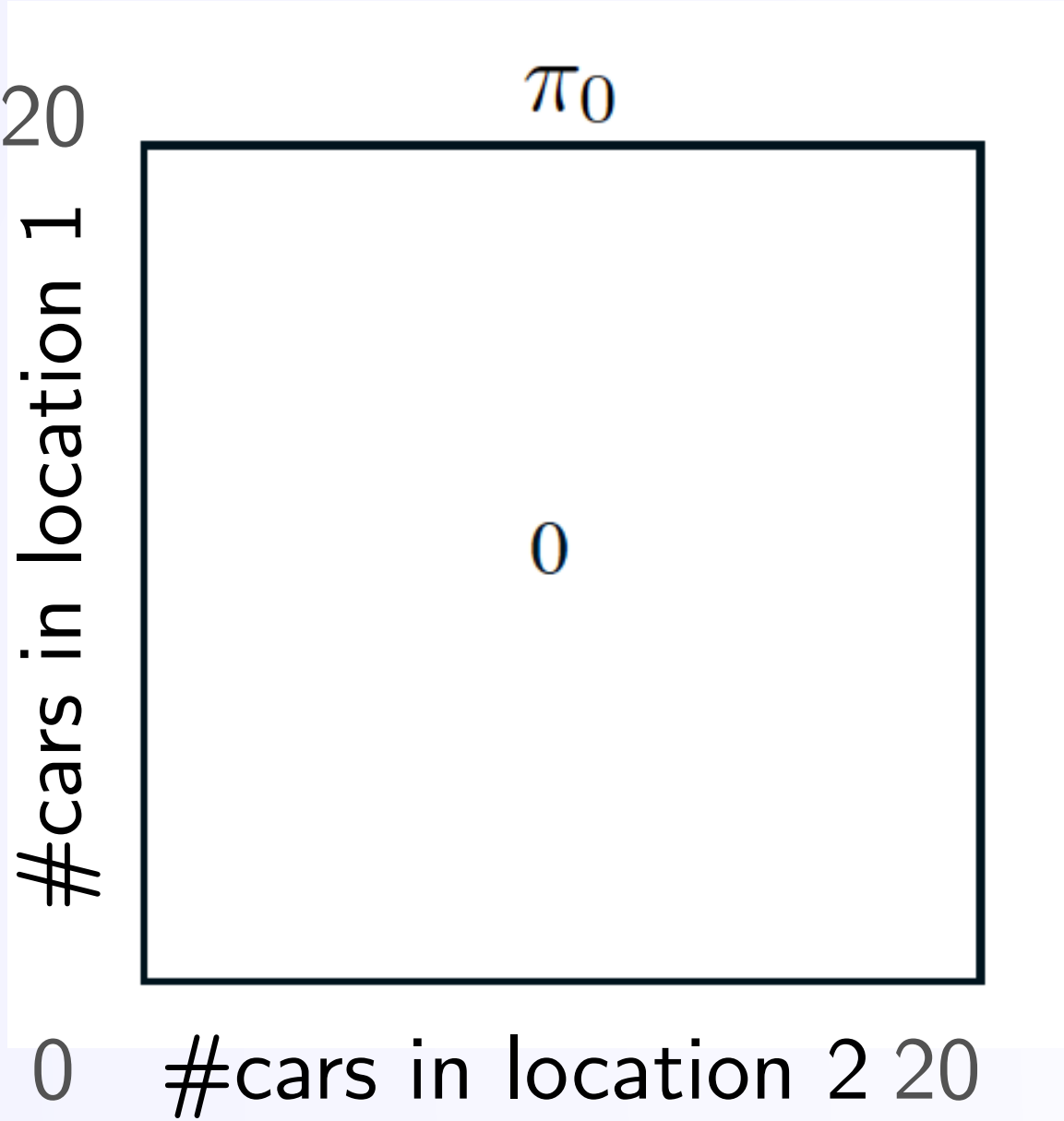
$$P[X = n] = \frac{3^n e^{-3}}{n!}$$

$$P[X = n] = \frac{4^n e^{-4}}{n!}$$

Actions: move 0 - 4 cars from one location to another overnight. Moving each car costs \$2 (reward -2)

- Customer comes
 - If available → Rents a car (Reward: \$10)
 - If not available → no business (no reward)
- Cars become available for rent the day after it is returned
- Maximum 20 cars in each location at any time
 - More than that → return to parent company
- Environment:
 - Number of customers at each location per day $\sim$ Poisson distribution
- Time steps: days
- States: the number of cars in each location (a pair of numbers) at the end of the day
- Continuing task with $\gamma = 0.9$ is a decent choice

Policy Iteration



Recall: we may arrive at the optimal policy before the evaluation converges 17

 $k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

	←	←	↕
↑	↖	↕	↓
↑	↕	↘	↓
↕	→	→	

 $k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

	←	←	↖
↑	↖	↖	↓
↑	↖	↘	↓
↖	→	→	

Already
optimal
policy

 $k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

	←	←	↖
↑	↖	↖	↓
↑	↖	↘	↓
↖	→	→	

Converged values

Value Iteration

- We may not need the policy evaluation to converge in order to improve the policy
- Idea: just run for one sweep (one iteration to update values of all states once), and update the policy
- Update rule (evaluation):

$$v_{k+1}(s) := \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

- Update rule (value iteration):

$$v_{k+1}(s) := \max_{a \in \mathcal{A}} \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

- This step implicitly improves the policy w.r.t. all states
- We can iterate the values this way and output a policy when it ends

Value Iteration

value_iteration(P, θ):

Initialize $v_T = 0$ and $v_i = \text{random}$ for all states $i \neq T$

$\delta = +\infty$

while $\delta > \theta$:

$\delta' = 0$

for $s \in \mathcal{S}$:

$v_s^{old} = v_s$

$v_s = \max_{a \in \mathcal{A}} \sum_{s', r} p(s', r | s, a) [r + \gamma v_{s'}]$ # updates values for all states

$\delta' = \max(\delta', |v_s^{old} - v_s|)$

$\delta = \min(\delta, \delta')$

Now output a deterministic policy

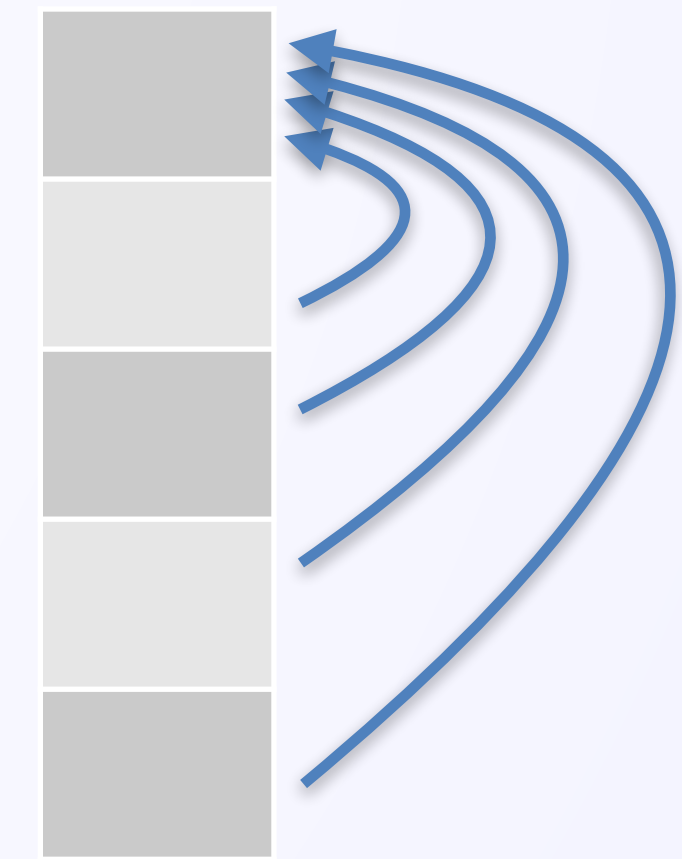
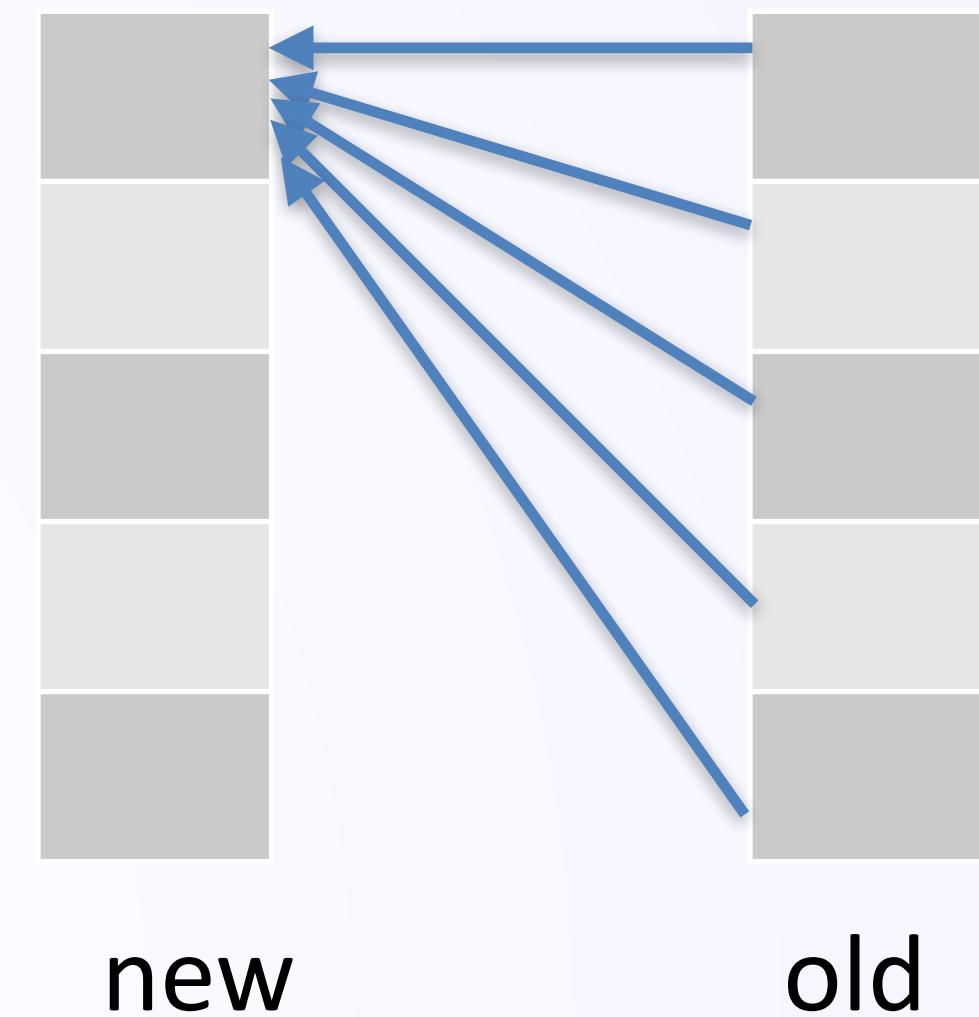
$\pi(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{s'}]$

Similarly, k sweeps (for $k > 1$) of policy evaluation per policy improvement step can be implemented to achieve faster convergence in policy iteration

one sweep
over all
states

Synchronous and asynchronous policy evaluation

- Synchronous: keep two arrays to store $v_k(s)$ for all $s \in \mathcal{S}$
 - Old values are used to compute all new values
 - Once all new values are computed, use the new array as old
- Asynchronous: keep one array, in place updates
- Example: suppose we have 5 states, $s = 1, 2, 3, 4, 5$ and $v_0(s) = 0$
- Synchronous:
 - $(v_0(1), v_0(2), v_0(3), v_0(4), v_0(5)) \rightarrow v_1(s)$ for all s (one sweep)
 - $(v_1(1), v_1(2), v_1(3), v_1(4), v_1(5)) \rightarrow v_2(s)$ for all s , and so on
- Asynchronous:
 - $(v_0(1), v_0(2), v_0(3), v_0(4), v_0(5)) \rightarrow v_1(1)$
 - $(v_1(1), v_0(2), v_0(3), v_0(4), v_0(5)) \rightarrow v_1(2)$
 - $(v_1(1), v_1(2), v_0(3), v_0(4), v_0(5)) \rightarrow v_1(3)$, and so on (in any order)
- Asynchronous must continue to update all states to converge correctly
- In practice converges much faster



asynchronous

References

- Sutton, R.S. and Barto, A.G., 2018. Reinforcement learning: An introduction. MIT press. <http://incompleteideas.net/book/the-book.html> (primary reference), Chapter 3
- StackExchange: [Why does policy evaluation converge?](#)